

BLUE JAY NIGHT RIDE

A practical guide to dispatch optimization with aggregate data

Research design, implementation instructions, data contracts and a value-entry workbook for Team FourSight.

Research question

Can coordinating assignments across pending requests serve more passengers within pickup and ride-time limits, using the same fleet?

Version 1.0 | 22 September 2026 | Team discussion and implementation specification

Our only operational source is the April 2026 aggregate report. We will construct several plausible demand scenarios, compare dispatch policies under identical conditions, and identify when coordination helps or hurts. We are not reconstructing individual historical trips.

What this document is - and is not

This is a proposed technical specification. It does not contain newly digitized PDF values, verified service rules, implemented simulation code, or experimental results. Numeric choices described as pilot settings are examples for team review, not Hopkins facts. Existing repository plans that expect future trip records are superseded by the aggregate-only scope here.

How teammates should use it

Read pages 2-3 together. Assign owners on page 12. Enter evidence and values using pages 4-6 and the companion JSON template. Build in the order on page 12, and do not interpret policy results until the verification gates on page 10 pass.

Navigation: 2 Scope and reasoning; 3 experiment; 4 extraction; 5 schemas; 6 parameter register; 7 scenario generation; 8 simulator; 9 optimizer; 10 validation; 11 evaluation; 12 team workflow; 13 configuration and references.

02 / TEAM IMPLEMENTATION GUIDE

The project in one page

Observed summaries → plausible request streams → one shared simulator → fair policy comparisons → a conditional recommendation.

The operational decision

At each dispatch event, which vehicle should serve each waiting request, and in what pickup/drop-off order? A second decision is whether to wait briefly for additional requests before committing. We will distinguish assignment quality from the effect of waiting.

What counts as success

A useful outcome may be improved service, a rule identifying when coordination is worthwhile, or evidence that a simpler method performs just as well. A positive improvement is not guaranteed. The practical deliverable is a candidate policy and its failure conditions, suitable for consideration in a future operational pilot.

In scope	Outside the initial scope
Aggregate audit; broad geographic zones; real road costs	Recovery of actual rider histories or the incumbent policy
Several demand, fleet and travel-time scenarios	A single supposedly correct synthetic history
Pooling-capable baseline and joint assignment	Building-by-building student activity reconstruction
Passenger service, modeled overflow, computational feasibility	Guaranteed Lyft savings, real vehicle control, deployment
Checks on spatial service differences	Demographic fairness claims without demographic evidence

Four evidence labels used throughout

Label	Meaning and example
OBSERVED	Read from a source with its definition and extraction uncertainty.
DERIVED	Computed from compatible observations, e.g. passengers per completed request.
ASSUMED	Chosen for a scenario, e.g. campus departures are stronger early.
SIMULATED	Produced by the model, e.g. waiting time under policy B.

External geography and road estimates receive their own source and retrieval date. They improve geographic or travel-time realism; they do not establish where students requested trips at a particular hour.

03 / TEAM IMPLEMENTATION GUIDE

Design the comparison before coding

Policy	Dispatch trigger	Assignment method
A: immediate greedy	Each request arrival; shared urgent-request rule	Best feasible pickup/drop-off insertion, one request at a time.
B: periodic greedy	Every W seconds; shared urgent-request rule	Same insertion rule, processing pending requests oldest first.
C: periodic joint	Same W and urgent rule as B	Choose compatible routes for multiple vehicles and requests jointly.

B versus A: tests the effect of periodic coordination with a fixed greedy method. **C versus B:** tests joint assignment at the same window. **C versus A:** measures the total intervention, but cannot alone explain which component helped.

Pilot settings to review, not operational facts

A small pilot can use $W = 30, 60$ and 120 seconds, with A as a separate immediate policy. Do not call $W = 0$ the joint method: it is a different algorithm. Add an event-triggered joint policy later only if meaningful uncommitted decisions remain to optimize.

Rules held fixed across policies

Use the same request stream, vehicle shifts, capacities, road model, starting state, service limits, rejection rules and objective priorities. Freeze the same committed movements. Policies may see current state and previously revealed requests, never future arrivals. Give B and C the same eligibility for reconsidering previously assigned but uncommitted requests.

Urgent-request behavior

At every new request and dispatch tick, calculate remaining pickup slack using a feasible insertion estimate. Requests that cannot safely wait until the next tick invoke a shared immediate greedy fallback. The margin is a named parameter. Count override events; frequent overrides may mean the window is operationally ineffective.

First pilot questions

Does each algorithm create legal pooled routes? Does B add waiting without gains? Does C find better combinations than B? How often does the optimizer fall back or time out? Run contrasting scenarios before scaling experiments.

Gate: agree on the primary metric, dispatch semantics, reconsideration rules and shared emergency fallback before implementing policy C. Otherwise an apparent gain may be a difference in rules.

04 / TEAM IMPLEMENTATION GUIDE

Extract evidence and reconcile definitions

Create a source manifest first. For each chart, record title, PDF page, reporting period, unit, population/status filter, aggregation grain and anything unknown. Do not rely on numbers quoted in earlier conversations.

Chart family	Enter into	Checks before downstream use
Daily requests by status	observations table: total/completed/cancelled/no-show/denied	Are categories exclusive? Are totals given or reconstructed? Can repeat bookings occur?
Hourly completed rides	observations: hour and completed count	Determine whether the hour refers to request, pickup or completion. Unknown remains unknown.
Origin/destination maps	cluster observations and zone allocations	Read counts; assess crop/occlusion and positional uncertainty. Do not assume each marker is a building.
Passengers; van/Lyft summaries	observations with source-system tags	Check trips versus passengers and dates before forming ratios.
Fleet by hour	observations with service coverage	Resolve or scenario-test all-services versus Homewood coverage and reporting days.
Wait/ride summaries	observations with quantile and unit	Check served population and passenger/request weighting. Do not infer a full distribution from a few quantiles.

Extraction procedure

1. Calibrate chart axes and units. 2. Read values with intervals where pixels limit precision. 3. Independently re-read selected values, peaks and anomalies. 4. Compare visible tooltip anchors where available. 5. Preserve the original observations when later reconciliation changes interpretation.

Reconciliation ledger

For each proposed identity, record left-hand total, right-hand total, residual, coverage compatibility and action. Examples: daily versus hourly completed totals; passenger series versus van ridership; van plus Lyft versus reported combined total. A discrepancy may be a definition issue, not measurement noise.

Do not force equality: keep an explicit unknown/outside-map component where warranted. Never allocate unexplained residuals to convenient zones just to make the generator balance. If filters remain unknown, model alternative interpretations.

05 / TEAM IMPLEMENTATION GUIDE

Data contracts and file locations

The following paths are proposed interfaces, not files that already exist. Keep restricted source material and generated operational tables out of public commits according to repository policy. All records need a stable ID and version.

Proposed path / row unit	Required fields	Who fills it
data/processed/observations.csv One chart observation	obs_id, chart_id, page, period, date_or_hour, metric, value, lower, upper, unit, population, evidence_status, notes	Data owner: source readings and uncertainty.
data/processed/zones.csv One common geographic zone	zone_id, name, geometry_ref, representative_lat/lon, category, uncertainty_note	Geography owner: approximate zones and sampled routable locations.
data/processed/zone_counts.csv Zone x endpoint x population	zone_id, endpoint, count, lower, upper, population, allocation_method, source_ids	Data + geography: reconciled mapping with provenance.
data/processed/travel_times.csv Directed pair x time band	origin_node, dest_node, time_band, seconds, meters, source, retrieval_date, model_version	Geography owner: road estimates; retain direction.
data/synthetic/requests.parquet One request	request_id, scenario_id, replicate, service_date, arrival_s, origin_node, dest_node, party_size, patience_s	Generator only. No service outcomes here.
outputs/runs/.../request_results.parquet One request x policy run	request_id, run_id, terminal_status, assigned_s, pickup_s, dropoff_s, vehicle_id, wait_s, ride_s, failure_reason	Simulator only; absent outcomes are null, never zero.
outputs/runs/.../run_manifest.json One policy run	run_id, config_hash, scenario_id, seed_set, policy, code_version, solver_settings, input_versions	Runner: supports exact reproduction.

Store vehicle schedules separately: vehicle_id, capacity, start_s, end_s, initial_node. Store event logs separately: event_id, simulation_time, event_type, vehicle_id, request_id, location and load. The event log is essential for debugging impossible results.

Time convention: seconds since the chosen service-night start internally; America/New_York and ISO dates in metadata. Define after-midnight attribution explicitly. Never mix minutes and seconds in model inputs.

06 / TEAM IMPLEMENTATION GUIDE

Parameter register: where values go

Fill the companion **team_config.template.json**, then save an experiment-specific copy. A null value means unresolved, not zero. Numerical choices need a source or an assumption rationale in the decision log. Every required null must be resolved before a main run.

Config key	Unit / value to enter	Basis and owner
service.start_local / end_local	Local HH:MM; initially null	Data owner checks report coverage and defines service night.
service.wait_limit_s / ride_limit_s	Seconds; initially null	Team verifies applicable rules. Earlier 20/25 minute discussion is provisional.
fleet.schedule_path	Path to vehicle-level schedule; null	Simulation owner defines plausible fleet scenarios. Do not silently use all-services counts.
demand.daily_totals_path	Path to extracted daily totals; null	Data owner specifies request population and repeat-booking caveat.
demand.hourly_request_weights	Nonnegative vector summing to 1; null	Model owner supplies alternative assumptions; not copied as fact from hourly completions.
demand.party_size_probs	Discrete probabilities; null	Use compatible passenger/request ratio if available; a mean alone does not identify probabilities.
demand.patience_spec	Distribution, parameters, limits; null	Scenario assumption. Independent of policy for paired runs.
spatial.beta_per_min / scenario_path	Distance sensitivity; scenario table; null	Model owner varies gravity and time-direction assumptions.
travel.matrix_path / multiplier	Path; positive factor; null	Geography owner supplies road model and congestion cases.
simulation.pickup_dwell_s / dropoff_dwell_s	Seconds; null	Simulation owner: assumptions. Vehicle hold limit is not automatically average dwell.
dispatch.windows_s / urgent_margin_s	[30,60,120] pilot / null	Team selects windows and shared urgency margin.
solver.time_limit_s / max_new_per_route	Seconds / integer; null	Optimization owner sets based on benchmark runtime.
evaluation.replicates / seed_base	Positive integer / integer; null	Analysis owner sets after pilot variability and seed design.

Also decide: travel-time randomness, allowable reassignment, overflow trigger, shift-end rule, horizon clearing, scenario weights, service guardrails and primary comparison. Record these before experiments; do not bury them in code.

07 / TEAM IMPLEMENTATION GUIDE

Construct the demand scenarios

Step 1 - reconcile spatial evidence

Place origins and destinations on a common coarse zone system. Broad and close-up maps may overlap; do not double-count them. Use uncertain allocations when cluster footprints are not known. A map count is not proof of its exact spatial extent or population.

Step 2 - create candidate O-D-time weights

For origin o , destination d and hour h , use a positive seed such as:

$$\text{seed}[o,d,h] = \text{base}[o,d] * \exp(-\text{beta} * \text{travel_min}[o,d,h]) * \text{activity_weight}[o,d,h]$$

The activity weight represents an explicitly chosen directional scenario. Compare early campus departures, early campus arrivals and mixed flows. Include feasible intra-zone trips where relevant; coarse zones can contain legitimate journeys. Exclude impossible pairs with documented structural zeros, not merely inconvenient ones.

Step 3 - balance compatible margins

For compatible populations, use iterative scaling to match origin totals, destination totals and the selected hourly distribution. Stop when residuals meet a declared tolerance. An average-zero directional adjustment alone does not guarantee these margins. Infeasible support must produce a diagnostic, not silent redistribution.

If maps describe completed trips, their margins constrain baseline served outputs, not automatically all generated requests. Either evaluate both population interpretations or use the maps as an explicit approximate spatial prior and report baseline mismatch. Label which interpretation each scenario uses.

Step 4 - sample request streams

Choose a daily request count from compatible observations or an explicitly scaled scenario. Allocate requests to hours using assumed arrival shares, then draw timestamps under smooth or bursty alternatives. Sample O-D pairs, routable endpoints, party sizes and patience. Use separate reproducible random streams for each component.

Step 5 - distinguish population targets from sample variation

A probability table can match target margins while a sampled night differs by chance. For exact synthetic monthly margins, use constrained integer allocation; otherwise report sampling residuals. Do not require every night to reproduce a monthly spatial pattern exactly.

Keep input and outcome apart: observed wait and pooled ride-duration summaries are baseline checks. They are not independent request attributes to draw and then reuse as simulated service results. Road time is not pooled ride time.

08 / TEAM IMPLEMENTATION GUIDE

Simulator behavior and accounting

Event loop

```
advance vehicles to next event time
apply arrivals, shift changes, pickups and dropoffs
update pending requests and remaining route feasibility
invoke dispatch if arrival/tick/urgency rules require it
commit allowed decisions and schedule next events
log state transitions; repeat until the night clears
```

Specify deterministic ordering for simultaneous events. For example, process drop-offs before pickups, then new requests, then patience/deadline checks and dispatch, with explicit equality rules. Use the same ordering for every policy.

Request states

An active request may be pending, assigned or onboard. Each request ultimately has exactly one terminal status: **completed, abandoned, modeled overflow, or unserved at close**. State changes require a reason code. An assigned rider can abandon before pickup if that is the shared patience rule; onboard riders cannot simply disappear.

Outcome definitions

Modeled overflow: an explicit outside-option decision under the chosen fallback rule. A request being infeasible at one instant is not necessarily proof it cannot be served later. Specify whether it remains pending until a deadline, is handed off immediately, or uses a forecast threshold.

Abandonment: a rider leaves before pickup according to the scenario patience model. This is not automatically identical to reported cancellations. Administrative cancellations, duplicates and no-shows may not be represented; document that gap.

Vehicle commitments and end of service

Freeze travel already underway and protect all onboard passengers. Define whether the next stop is frozen or only the current road movement. At shift end, stop assigning new work according to the shared rule and complete accepted service. Continue simulation beyond the last arrival long enough to clear accepted trips; account for residual requests explicitly.

Constraints checked after every route change

Pickup precedes drop-off; each party stays on one vehicle; load remains between zero and capacity; all travel has a valid directed cost; service windows and existing commitments remain feasible. Distinguish projected feasibility at assignment from actual attainment if travel times are stochastic.

Accounting identity: generated requests = completed + abandoned + modeled overflow + unserved at close. Check passenger-weighted totals separately. No request may appear in two terminal categories.

09 / TEAM IMPLEMENTATION GUIDE

A pooling-capable joint optimizer

A practical first MILP can select from precomputed feasible route options. Each option is an ordered sequence of pickups and drop-offs for one vehicle, retaining its required stops and adding one or more pending requests. Feasibility is checked by simulating that sequence from the current state.

Notation and model

Let $R(v)$ be the candidate routes for vehicle v . Let $x[v,r]$ be a binary route-selection variable. Let $a[q,v,r]$ indicate whether route r serves pending request q . Let $y[q]$ indicate that q is selected for service now, and $p[q]$ be its party size.

$$\begin{aligned} \sum_r x[v,r] &= 1 && \text{for each vehicle } v \\ \sum_{v,r} a[q,v,r] * x[v,r] &= y[q] && \text{for each request } q \\ x[v,r], y[q] &\in \{0,1\} \end{aligned}$$

Priority 1: maximize $\sum_q p[q] * y[q]$
 Priority 2: minimize added waiting, then added travel
 while preserving the best Priority 1 value

Include a no-new-request route for every vehicle. Existing commitments must be served in every eligible option or fixed explicitly. Unselected requests remain pending unless the common overflow or abandonment rule fires. The myopic objective supports the experiment but does not guarantee maximum service over the entire night.

Construct route options carefully

Enumerate feasible insertions and small combinations of new requests, using a configurable cap and pruning. Include options with multiple new requests on the same vehicle; otherwise the experiment cannot test joint pooling. Record the candidate-generation limits and route counts. Where possible, include the greedy solution as a candidate incumbent.

Objective and fairness controls

Use the same service-first priorities and stable tie-breaking for greedy insertion. If passenger weighting disadvantages single riders, report request-level service rates and consider a separate sensitivity case. Do not change objective weights after seeing which policy wins without marking the analysis exploratory.

Computation is part of the policy

Record solve time, incumbent quality and timeouts. Specify whether computational delay enters simulated time; an idealized zero-delay model must be labeled. A time-limited feasible incumbent may be used; otherwise invoke the documented greedy fallback. Bounds apply only to the restricted candidate model, not to the entire routing problem.

Prototype warning: ordinary request-to-vehicle Hungarian matching selects at most one new request per vehicle per round. It is not equivalent to this route-combination model.

10 / TEAM IMPLEMENTATION GUIDE

Verification, calibration and release gates

Gate	Required evidence	If it fails
G1: input integrity	Definitions and coverage recorded; reconciliation residuals explained or retained; units verified.	Do not combine incompatible series. Carry alternative interpretations.
G2: simulator correctness	Hand-solvable cases: one request; compatible pooling; capacity conflict; urgent deadline; unreachable pair; shift end; abandonment.	Fix engine and accounting before comparing policies.
G3: policy parity	Same inputs, commitments, eligibility, service rules and information; matched B/C windows.	Treat comparisons as invalid until aligned.
G4: baseline plausibility	Compare compatible completion and hourly patterns plus wait/ride summaries, with source uncertainty.	Diagnose model/population mismatch; narrow claims.
G5: experimental reliability	Paired repetitions; uncertainty intervals; runtime logs; scenario sensitivity.	Increase justified repetitions or report uncertainty.
G6: recommendation	Fresh test runs for any selected rule; harms and failure conditions reported.	Present exploratory findings rather than a deployment recommendation.

Calibration protocol

Before fitting, list which summaries are calibration targets and which are held-out checks. Fit or choose only a small, declared parameter set. Retain multiple plausible parameter combinations if the data cannot distinguish them. Freeze them before policy comparisons. Apply each retained configuration to every policy.

Limits of held-out checks

With one month and shared aggregates, held-out summaries are not fully independent evidence. Passing them makes the model more plausible but does not establish true demand, the real dispatch policy or expected real-world savings. A baseline mismatch may be tolerable for a method experiment, but it limits Hopkins-specific interpretation.

Do not validate by construction

Reproducing margins enforced by balancing is an integrity check. It is not behavioral validation. Matching calibrated waiting summaries is not a prediction. A different dispatch policy should be allowed to change completion, waiting and overflow outcomes; never force those historical outcomes on every policy.

11 / TEAM IMPLEMENTATION GUIDE

Metrics, uncertainty and interpretation

Metric	Definition and reporting rule
Primary: timely passengers served	Sum party sizes for completed requests satisfying both wait and ride limits. Report count and share of all requested passengers.
Request service rate	Completed requests / all generated requests. Also report timely completed requests / all requests.
Waiting and riding	Wait = pickup - arrival; ride = dropoff - pickup. Report quantiles among served riders with weighting stated. Pair with unserved metrics to expose selection effects.
Modeled overflow / abandonment	Separate counts and rates with all requests as denominator. Never automatically label these Lyft trips or real cancellations.
Service by area	Origin/destination-zone service rates, waits and sample sizes. Avoid conclusions from tiny groups.
Vehicle and solver performance	Occupied, empty and idle time; end-of-shift overtime; runtime percentiles, feasible-solution and fallback rates.

Paired comparisons

For each scenario and replicate, reuse the same request stream and exogenous environment across policies. Pre-generate randomness by request ID and time/location where appropriate; one shared seed is insufficient if policies consume random draws in different orders. Compute policy differences within each paired night.

Report mean differences and uncertainty intervals across independent replicate pairs within a scenario. Use a paired bootstrap or a justified alternative. Choose replication count from pilot variability and a team-defined practical effect size. Simulation intervals represent random variation within assumptions, not all model uncertainty.

Guardrails and policy selection

Enter the minimum meaningful gain and maximum acceptable service deterioration before the main experiment. If limits are hard in a deterministic model, compliant served trips should satisfy them by construction; emphasize service coverage and wait distributions too. Without justified scenario probabilities, show scenario-specific results rather than an arbitrary weighted average.

Final figures

Produce an aggregate audit plot, scenario map, wait-versus-service tradeoff, policy-by-scenario improvement table and runtime summary. Show losses and null results. Separate random variation from differences across fleet, timing, patience and travel assumptions.

Permitted conclusion: "Policy C improved timely service in these tested scenarios, under these assumptions."
Unsupported conclusion: "Hopkins will save this percentage of Lyft trips."

12 / TEAM IMPLEMENTATION GUIDE

Team workflow and decision worksheet

Work package / suggested owner	Deliverable	Completion gate
1. Data steward Name: _____	Observation tables, dictionary, reconciliation ledger; aggregate diagnostics in parallel.	G1 passes or unresolved items have explicit downstream scenarios.
2. Geography + demand Name: _____	Zones, road matrix, scenario generator and assumptions.	Reproducible streams; feasible geography; margins and population interpretation documented.
3. Simulation engineer Name: _____	Event engine, accounting, greedy policies, logs.	G2 and G3 pass on hand-checkable cases.
4. Optimization + evaluation Name: _____	Candidate-route optimizer, pilot experiments, sensitivity and paired analysis.	G4-G6 assessed; all limitations reported.

Roles can be shared, but each artifact needs one accountable owner and a different reviewer. Work packages 1 and 2 can progress together after the source manifest; optimization can use tiny artificial instances while the demand model is being built.

Suggested notebooks - interfaces to later implementation

00_source_audit.ipynb	observations + reconciliation
01_aggregate_diagnostics.ipynb	profiles + anomalies
02_zones_and_travel.ipynb	geography + directed costs
03_scenario_generator.ipynb	demand + construction checks
04_simulator_checks.ipynb	invariant and example checks
05_policy_pilot.ipynb	timing + assignment contrasts
06_main_experiments.ipynb	paired runs + metrics
07_sensitivity_and_report.ipynb	assumptions + recommendation

First team meeting: decisions to fill in

Primary comparison: _____ Owner: _____
 Service rules source: _____ Reviewer: _____
 Initial geographic extent: _____
 Required pilot scenarios: _____
 Practical improvement / harm thresholds: _____
 Pilot review date: _____ Main run freeze date: _____

Use gates rather than an assumed semester calendar. If behind, cut fine spatial detail, optional place enrichment and extra policy windows first. Keep correct pooling, fair comparisons, burst sensitivity where relevant, accounting checks and honest uncertainty reporting.

13 / TEAM IMPLEMENTATION GUIDE

Configuration, decisions and references

How to enter values

The companion template is `docs/technical/team_config.template.json`. Copy it to a named experiment configuration before filling it. JSON has no comments, so rationale belongs in a separate decision log. Paths in the template are proposed interfaces; input datasets still need to be produced.

```
Example decision-log record (illustrative only):
parameter: dispatch.urgent_margin_s
value: 30
evidence_status: ASSUMED
reason: pilot buffer before the next dispatch opportunity
alternatives: [0, 30, 60]
owner: TEAM_MEMBER
reviewer: TEAM_MEMBER
decision_date: YYYY-MM-DD
status: proposed
```

The value 30 above is an example of documentation, not a recommended safety margin. For observations, replace the rationale with a source page and extraction interval. For verified rules, record the official source and applicable date. Required unresolved values must cause configuration validation to fail.

Before launching a main experiment

Confirm that required fields are populated; probabilities sum to one; units and date conventions match; input paths and versions exist; every scenario has a rationale; baseline checks are recorded; policy objectives are fixed; random streams are paired; solver fallback works; output directories are unique; and the code/config/input versions are saved.

Sources and scope of verification

Operational source to extract: *HW Night Ride Data April 2026.pdf*, provided in the repository. This guide specifies an extraction workflow; it does not independently certify chart values or service targets.

Project context reviewed: repository README.md and the project-direction discussion through 22 September 2026. The README still describes potential trip-level acquisition; this guide adopts the user-confirmed constraint that no additional operational data will arrive.

Methods: the route-selection formulation, experimental controls and proposed schemas in this guide are a design specification. They are not a literature review or a claim of methodological novelty. Add primary research and verified tool documentation during implementation, before making publication or provider-capability claims.

Research disclosure: this guide was drafted with AI assistance. Team members should verify source readings, understand and review the model, and document AI use according to the course requirements. The deliverable has no simulated performance findings yet.

Next concrete milestone: a reviewed source manifest, a filled pilot configuration and one hand-verified pooled-route example. Once those exist, the team can build with a shared understanding of what is measured and what is assumed.